

Big Data Assignment :

Take two csv file

Load it in scala

Merge it

Do some use cases

Dataset

id,name,city,salary

1,Arun,Bangalore,5000

2,Ravi,Chennai,6000

3,John,Hyderabad,5500

4,Meena,Mumbai,7000

5,Kumar,Kochi,4500

id,name,city,salary

6,Asha,Delhi,6500

7,Ravi,chennai,4000

8,Mani,kochi,6000

9,Deva,Delhi,7500

10,Saran,Bangalore,7000

Load CSV Files

```
scala> import scala.io.Source
```

```
import scala.io.Source
```

```
scala> var file1=Source.fromFile("C:/Users/Administrator/Downloads/e1.csv")
```

```
var file1: scala.io.BufferedSource = <iterator>
```

```
scala> var file2=Source.fromFile("C:/Users/Administrator/Downloads/e2.csv")
```

```
var file2: scala.io.BufferedSource = <iterator>
```

```
scala> var d1=file1.getLines().drop(1).toList
```

```
var d1: List[String] = List(1,Arun,Bangalore,5000, 2,Ravi,Chennai,6000,  
3,John,Hyderabad,5500, 4,Meena,Mumbai,7000, 5,Kumar,Kochi,4500)
```

```
scala> var d2=file2.getLines().drop(1).toList
```

```
var d2: List[String] = List(6,Asha,Delhi,6500, 7,Ravi,chennai,4000, 8,mani,kochi,6000,  
9,Deva,Delhi,7500, 10,Saran,Bangalore,7000)
```

Merge Two CSV Files

```
scala> val merged=d1++d2
val merged: List[String] = List(1,Arun,Bangalore,5000, 2,Ravi,Chennai,6000,
3,John,Hyderabad,5500, 4,Meena,Mumbai,7000, 5,Kumar,Kochi,4500, 6,Asha,Delhi,6500,
7,Ravi,chennai,4000, 8,mani,kochi,6000, 9,Deva,Delhi,7500, 10,Saran,Bangalore,7000)

scala> val emps=merged.map{line=>
  | val col=line.split(",")
  | (col(0).toInt,col(1),col(2),col(3).toInt)
  | }
val emps: List[(Int, String, String, Int)] = List((1,Arun,Bangalore,5000), (2,Ravi,Chennai,6000),
(3,John,Hyderabad,5500), (4,Meena,Mumbai,7000), (5,Kumar,Kochi,4500),
(6,Asha,Delhi,6500), (7,Ravi,chennai,4000), (8,mani,kochi,6000), (9,Deva,Delhi,7500),
(10,Saran,Bangalore,7000))
```

Filter Employees with High Salary

```
scala> val highsal=emps.filter(_._4>=6000)
val highsal: List[(Int, String, String, Int)] = List((2,Ravi,Chennai,6000), (4,Meena,Mumbai,7000),
(6,Asha,Delhi,6500), (8,mani,kochi,6000), (9,Deva,Delhi,7500), (10,Saran,Bangalore,7000))

scala> highsal.foreach(println)
(2,Ravi,Chennai,6000)
(4,Meena,Mumbai,7000)
(6,Asha,Delhi,6500)
(8,mani,kochi,6000)
(9,Deva,Delhi,7500)
(10,Saran,Bangalore,7000)
```

Calculate Average Salary

```
scala> val avgsal=emps.map(_._4).sum/emps.size
val avgsal: Int = 5900

scala> print(avgsal)
5900
```

Find Maximum Salary

```
scala> val maxsal=emps.map(_._4).max
val maxsal: Int = 7500
```

```
scala> print(maxsal)
7500
```

Find Miniimum Salary

```
scala> val minsal=emps.map(_._4).min
val minsal: Int = 4000
```

```
scala> print(minsal)
4000
```

Group Employees by City

```
scala> var bycity=emps.groupBy(_._3)
var bycity: scala.collection.immutable.Map[String,List[(Int, String, String, Int)]] =
HashMap(Bangalore -> List((1,Arun,Bangalore,5000), (10,Saran,Bangalore,7000)), kochi ->
List((8,mani,kochi,6000)), Delhi -> List((6,Asha,Delhi,6500), (9,Deva,Delhi,7500)), Chennai ->
List((2,Ravi,Chennai,6000)), chennai -> List((7,Ravi,chennai,4000)), Kochi ->
List((5,Kumar,Kochi,4500)), Mumbai -> List((4,Meena,Mumbai,7000)), Hyderabad ->
List((3,John,Hyderabad,5500)))
```

```
scala> bycity.foreach{case(city,emp)=>println(city+":"+emp.size)}
Bangalore:2
kochi:1
Delhi:2
Chennai:1
chennai:1
Kochi:1
Mumbai:1
Hyderabad:1
```

Sort Employees by Salary

```
scala> var sorted=emps.sortBy(_._4)
var sorted: List[(Int, String, String, Int)] = List((7,Ravi,chennai,4000), (5,Kumar,Kochi,4500),
(1,Arun,Bangalore,5000), (3,John,Hyderabad,5500), (2,Ravi,Chennai,6000),
(8,mani,kochi,6000), (6,Asha,Delhi,6500), (4,Meena,Mumbai,7000), (10,Saran,Bangalore,7000),
(9,Deva,Delhi,7500))

scala> sorted.foreach(println)
(7,Ravi,chennai,4000)
(5,Kumar,Kochi,4500)
```

(1,Arun,Bangalore,5000)
(3,John,Hyderabad,5500)
(2,Ravi,Chennai,6000)
(8,mani,kochi,6000)
(6,Asha,Delhi,6500)
(4,Meena,Mumbai,7000)
(10,Saran,Bangalore,7000)
(9,Deva,Delhi,7500)